TryHackMe — Watcher Writeup

Overview

Watcher is a medium Linux box built on a chained exploit path: an LFI leaks FTP creds, an uploaded reverse shell lands on www-data, and four separate misconfigurations (a sudo rule, a writable cron script, a hijackable Python import, and a world-readable backup key) pivot that foothold all the way to root.

Flag	User gained	Technique
flag_1	pre-auth	Initial web recon
flag_2	ftpuser	FTP login with LFI-leaked credentials
flag_3	www-data	LFI-triggered reverse shell uploaded via FTP (or LFI2RCE via PHP filter chain)
flag_4	toby	`sudo -u toby /bin/bash`
flag_5	mat	Cron job (`cow.sh`) overwritten by toby
flag_6	will	Python library hijack (`cmd.py` imported by `will_script.py`)
flag_7	root	SSH private key leaked via `adm` group read access

www-data escalates to root through four pivots

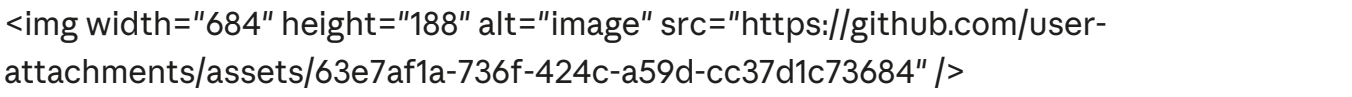


privilege-escalation chain · 4 pivots, www-data to root

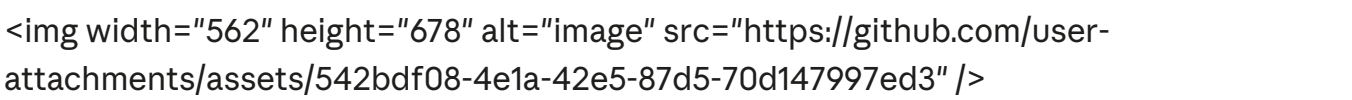
Each pivot abuses a different misconfiguration: a sudo rule, a writable cron script, a hijacked Python import, and a readable backup key.

1. Recon

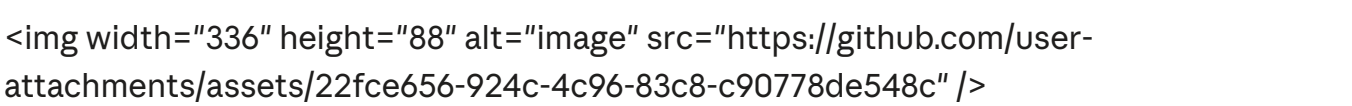
The server has 3 open ports: 21 (FTP), 22 (SSH), and 80 (HTTP).



Used `dirsearch` to fuzz directories:

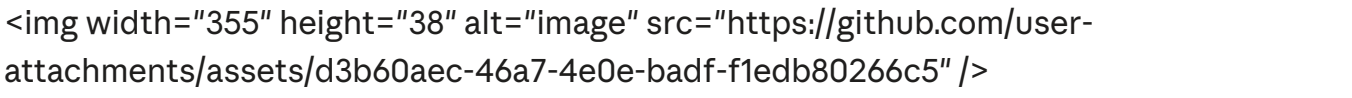


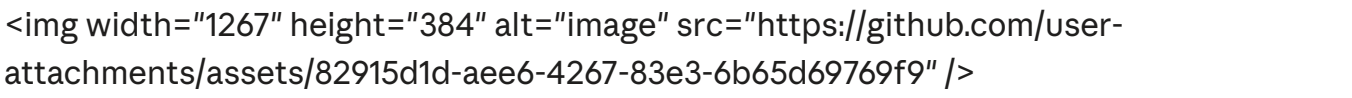
Contents of `robots.txt`:



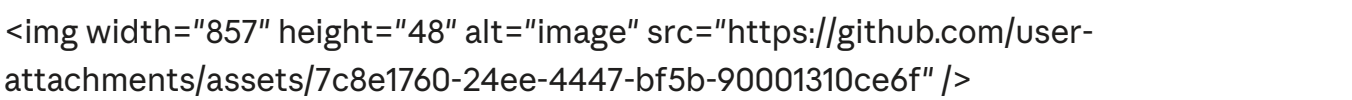
I tried accessing the `/secret_file_do_not_read.txt` endpoint, but got a 403 Forbidden response.

Going back to the website's homepage, I noticed the site is vulnerable to LFI (reference).





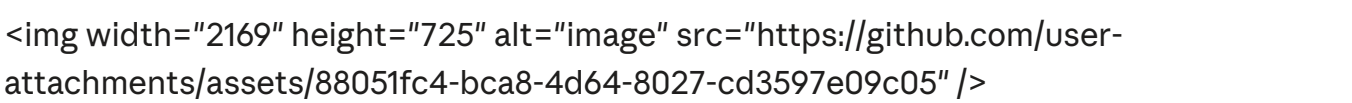
I then tried reading `secret_file_do_not_read.txt` via the LFI:



From the file's content, we obtain FTP credentials, and this user has write access and can store files in `/home/ftpuser/ftp/files`. We can abuse this to upload a reverse shell and trigger it via the LFI.

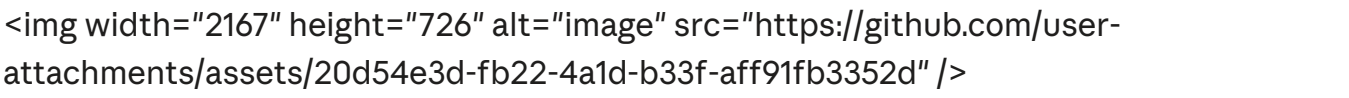
2. Exploit

2.1 Read flag_1



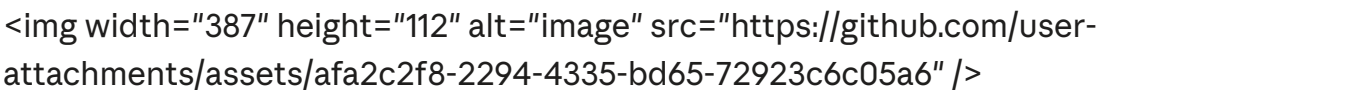
2.2 Read flag_2

Log into FTP with the credentials obtained above and read `flag_2.txt`:

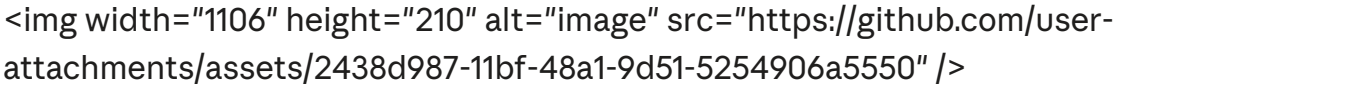


2.3 Read flag_3

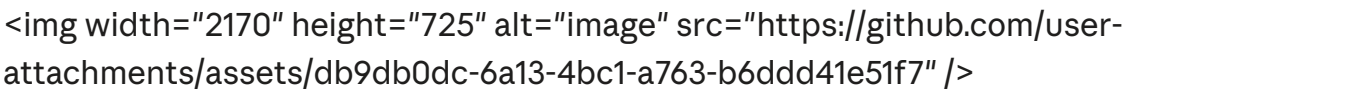
Navigate to the `files` directory and use the `put` command to upload a reverse shell:



Use the LFI to trigger the payload:

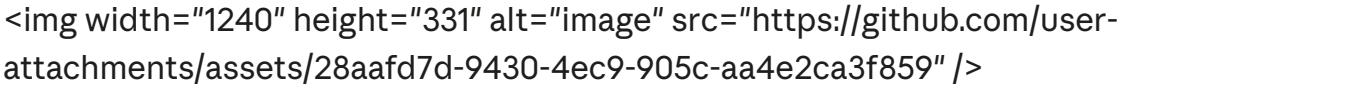


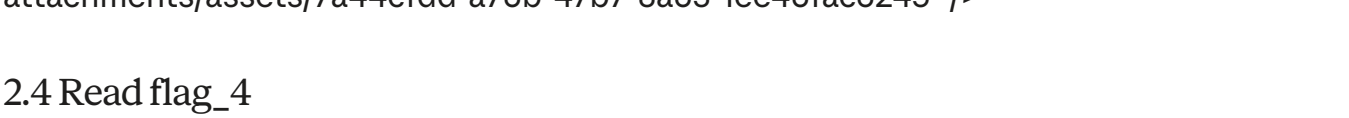
Use `find` to locate `flag_3.txt` and read it:



Alternatively, we can use the **LFI2RCE via PHP Filters** technique to get a shell back (reference).

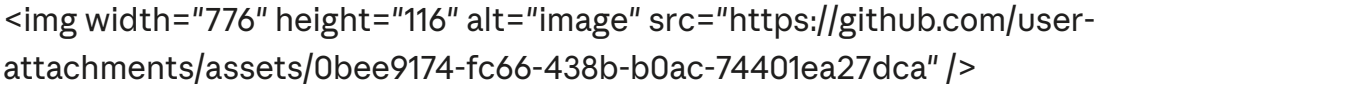
You can use synacktiv's tool to generate the payload: https://github.com/synacktiv/php_filter_chain_generator





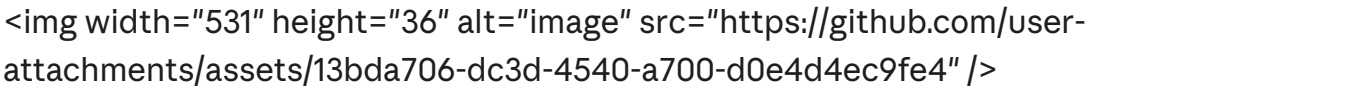
2.4 Read flag_4

Ran `sudo -l` to check for anything interesting:



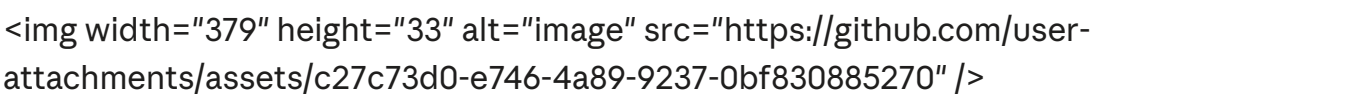
From here we can see that we can operate on the system as the `toby` user.

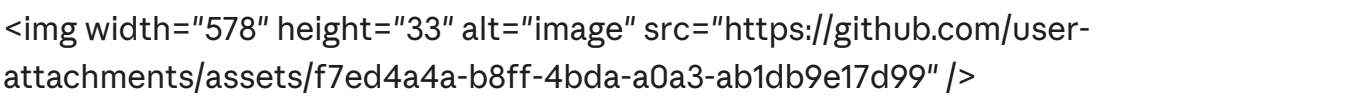
Reading the flag:



2.5 Read flag_5

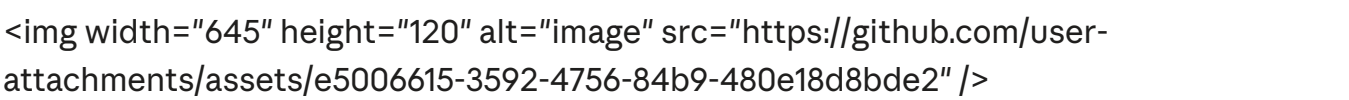
Checked `/etc/crontab`:



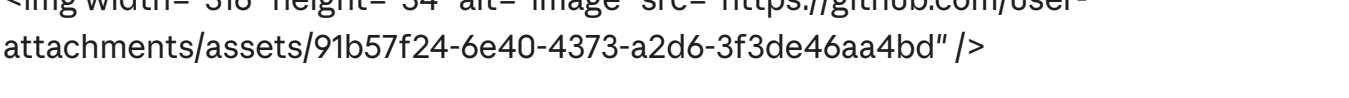


We can see that `mat` is scheduled to execute the `cow.sh` file.

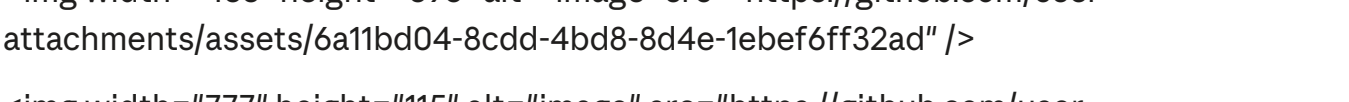
So we can abuse the `toby` account to modify `cow.sh` and obtain a new shell as `mat`:

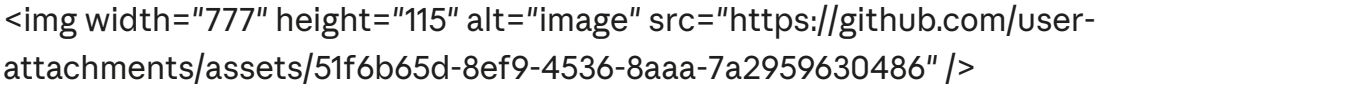


Reading the flag:



2.6 Read flag_6

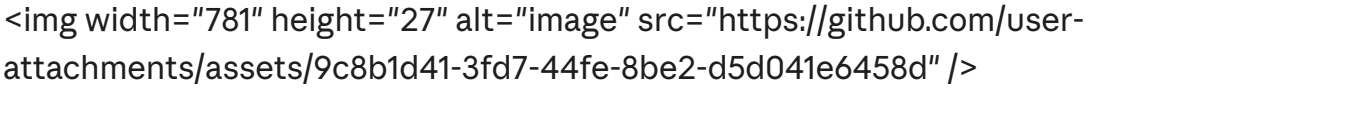


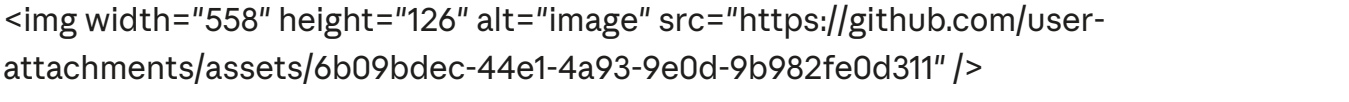


As before, we can execute `/home/mat/scripts/will_script.py` as the `will` user.

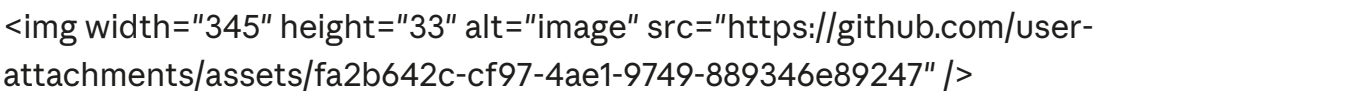
We can see that `will_script.py` imports `cmd.py`, and after checking, we find that the `mat` user has write access to `cmd.py`.

We can abuse this to inject a payload into `cmd.py`; when `will_script.py` imports and executes it, we get a shell as `will`:



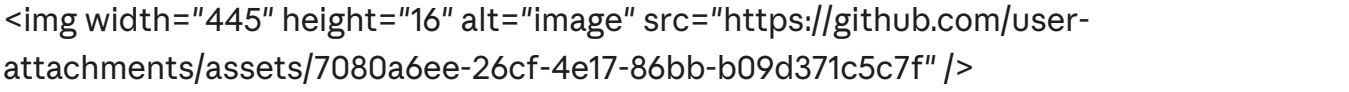


Reading the flag:



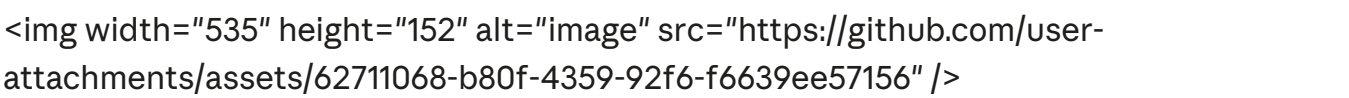
2.7 Read flag_7

Checked `id` for `will`:

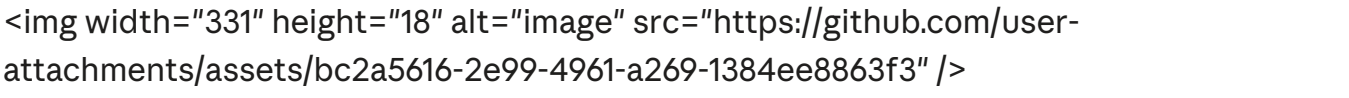


We can see that `will` belongs to the `adm` group, one of the special groups that can read certain sensitive directories.

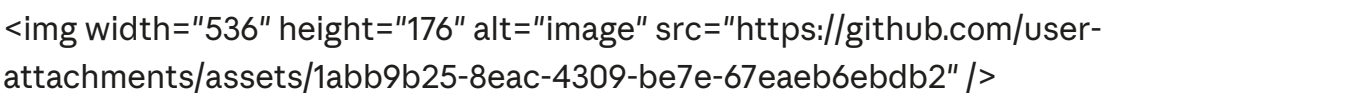
Used `find` to search for files:



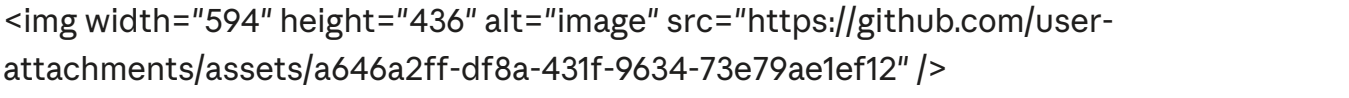
Here it looks like we found a file containing root's key:



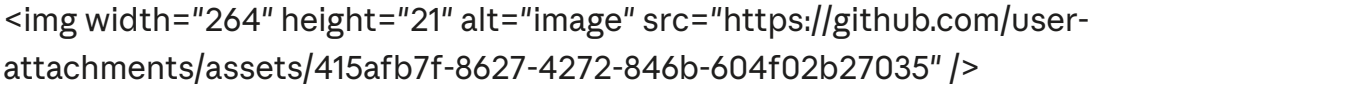
Reading the file:



Excellent — it looks like we found the private key used for root's SSH login:



Reading `flag_7.txt`:



Done!